SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 — ARCHITECTURE DESIGN ASSIGNMENT

Architecture Design Report

Intelligent AI-Based Smart Public Transportation Demand Forecasting System

Field	Details
Course Code	CSA1014
Course Title	Software Engineering
CO Assessed	CO2
Assessment Tool	Assessment Tool 2 — Architecture Design Assignment
Weightage	-
Student Name	N.RAMYA
Register Number	192511403
Date	4/8/26

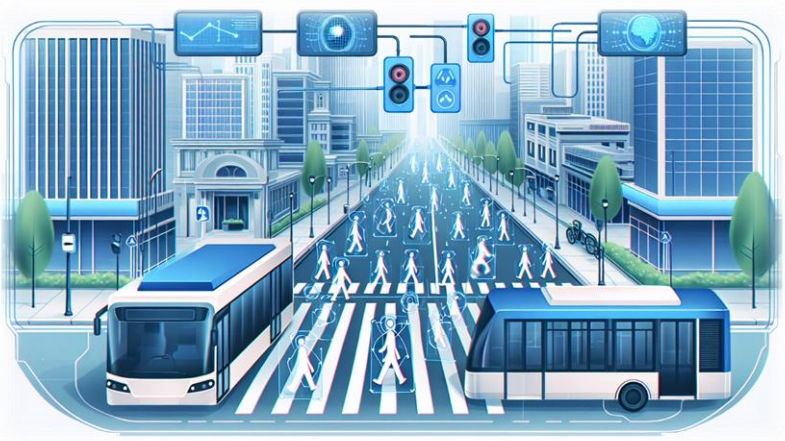

Table of Contents

1. Analysis of System Requirements

1.1 System Objectives (from the Problem Statement)

The Intelligent AI-Based Smart Public Transportation Demand Forecasting System must analyze historical ridership, weather, traffic, and event data to predict passenger demand; recommend optimized bus/train schedules; improve fleet allocation; and provide real-time passenger information. These objectives directly drive the choice of architecture, since the system must simultaneously handle high-volume real-time data streams (GPS/IoT), computationally intensive AI forecasting, and low-latency information delivery to thousands of concurrent users.

1.2 Functional Requirements Influencing the Architecture

Requirement	Architectural Implication
AI-based demand forecasting from multiple data sources	Requires an isolated, independently scalable compute-intensive service (favors microservices).
Real-time GPS/IoT passenger flow monitoring	Requires a high-throughput, low-latency streaming/event mechanism (favors event-driven architecture).
Schedule and fleet optimization recommendations	Requires services that can be updated/deployed independently as optimization algorithms evolve.
Real-time passenger-facing information	Requires a well-defined API layer decoupled from backend processing (favors client-server / API gateway pattern).
Integration with external weather/traffic APIs	Requires loosely coupled integration adapters that can change independently of core logic.
Analytics and reporting for multiple stakeholder roles	Requires a separate reporting/analytics component reading from a shared data store without impacting transactional services.

1.3 Non-Functional Requirements and Key Quality Attributes

The following quality attributes were identified as most critical to the architectural decision, based directly on the non-functional requirements defined during requirement analysis:

- Scalability — the system must scale independently for data ingestion (thousands of vehicles) and for user-facing traffic (thousands of passengers) without over-provisioning the entire system.
- Performance — demand forecasts and real-time arrival predictions must be delivered within a few seconds.
- Reliability — a failure in one component (e.g., the analytics module) must not bring down core services such as real-time passenger information.
- Availability — passenger-facing services must remain available 24/7, including during deployment of updates to other components.
- Security — role-based access control and encrypted communication are required across all client and service boundaries.
- Maintainability — the AI forecasting model and optimization algorithms will be updated frequently and must be deployable without redeploying the entire system.

1.4 Summary of Requirement-Driven Architectural Needs

Taken together, these requirements indicate that no single classical architectural style alone (e.g., a purely layered monolith) is sufficient. The system needs (a) independent scalability and deployability of compute-heavy AI components, (b) a real-time, asynchronous data pipeline for IoT/GPS telemetry, and (c) a well-structured client-server boundary for multiple types of front-end applications. This analysis directly motivates the hybrid architectural style selected in Section 2.

2. Selection of Suitable Software Architecture

2.1 Selected Architectural Style

A Hybrid Architecture combining Microservices, Event-Driven Architecture (EDA), and a Layered structure within each service is proposed for this system. A lightweight Client-Server / API Gateway pattern governs how front-end applications interact with backend services.

2.2 Justification

Architectural Style Considered	Suitability for This System
Pure Layered (Monolithic) Architecture	Simple to build, but the AI forecasting workload cannot be scaled independently from the rest of the system, and any single-layer failure risks the entire application. Rejected as the sole style.
Pure Client-Server	Adequate for basic request/response interactions but has no native mechanism for high-volume, continuous real-time IoT/GPS data streams. Rejected as the sole style.
Pure Microservices (without event-driven backbone)	Provides independent scalability and deployability, but synchronous REST-only communication between many services would create tight coupling and latency bottlenecks for real-time telemetry. Needs an event backbone.
Pure Event-Driven Architecture	Excellent for real-time data ingestion, but request/response interactions (e.g., passenger app queries) are more naturally and simply served synchronously. Needs a service layer.

Selected: Hybrid (Microservices + Event-Driven + Layered)	Combines the strengths of all three: microservices give independent scalability/deployability to the AI, scheduling, fleet, and analytics functions; the event bus (Kafka/MQTT) efficiently handles continuous IoT/GPS/weather/traffic streams; and each service is internally organized in layers (presentation/API, business logic, data access) for maintainability.

2.3 Why This Best Fits the Problem Statement

- The AI forecasting workload is computationally distinct from scheduling and fleet logic, so it benefits from being an independently scalable microservice.
- GPS/IoT sensor data arrives continuously and asynchronously, which is a natural fit for an event-driven backbone rather than repeated polling.
- Passenger, operator, fleet, and admin clients have very different interaction patterns and access rights, which are cleanly served through a single API Gateway that routes to the right microservice.
- Because CO2 explicitly requires designing a scalable architecture, a hybrid microservices/event-driven approach demonstrates the ability to combine architectural styles rather than force-fitting one classical pattern.

3. Software Architecture Diagram

The diagram below illustrates the complete structure of the proposed system: the client layer, the API gateway, the microservices with their individual data stores, the event bus that forms the real-time backbone, the data ingestion/integration layer, external systems, and the shared data platform. Arrow styles distinguish synchronous REST/HTTPS calls, asynchronous publish/subscribe events, and direct data read/write operations.

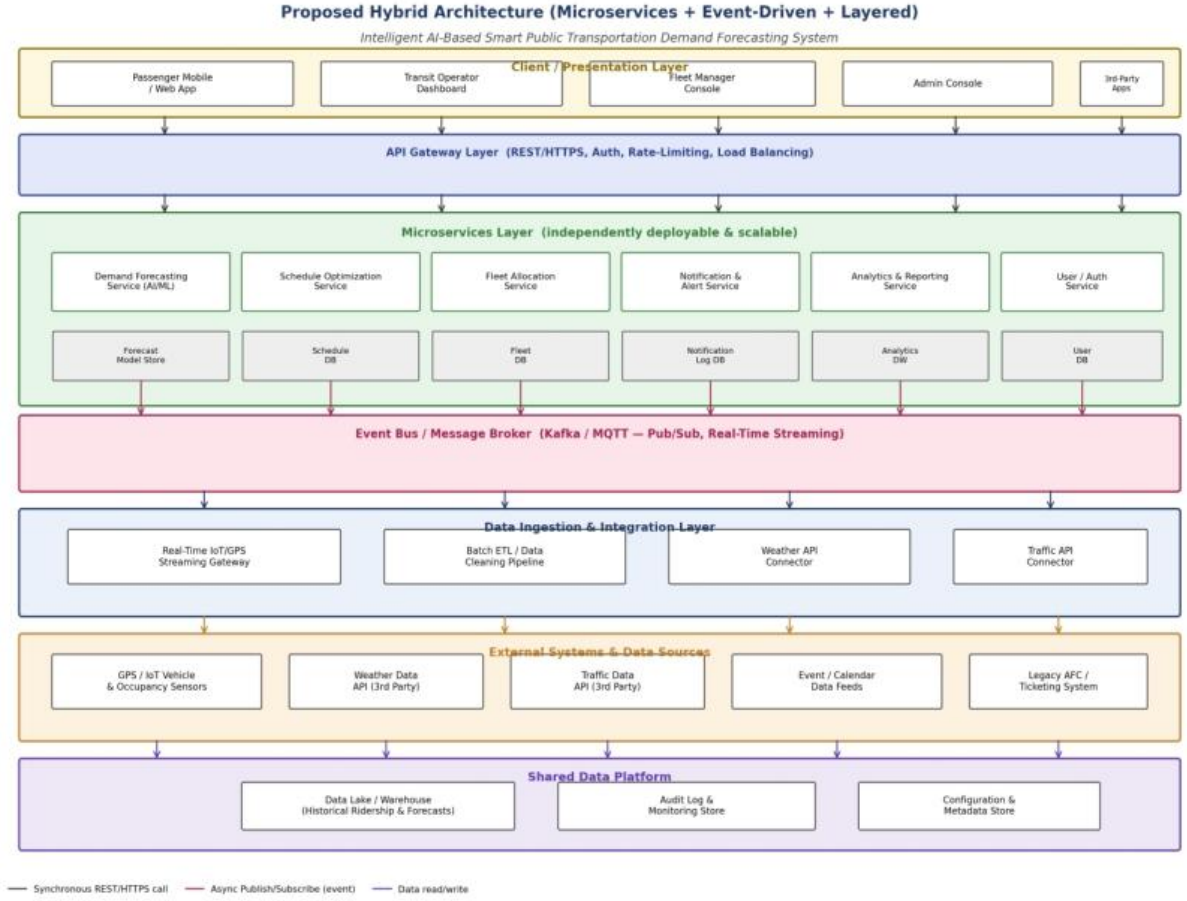

Figure 1: Proposed Hybrid (Microservices + Event-Driven + Layered) Architecture

4. Components and Their Interactions

4.1 Component Responsibilities

Component	Type	Responsibility
Client / Presentation Layer	Passenger app, operator dashboard, fleet console, admin console	Provide role-specific user interfaces; consume APIs; render real-time and historical data.
API Gateway	Single entry point for all client requests	Handles authentication, authorization, rate-limiting, request routing, and load balancing to backend microservices.
Demand Forecasting Service	AI/ML component	Consumes historical, weather, traffic, and event data; produces demand predictions with confidence scores; publishes/serves forecasts to other services.
Schedule Optimization Service	Business logic component	Consumes forecasts; computes optimized trip frequency/timing; exposes recommended schedules for operator approval.
Fleet Allocation Service	Business logic component	Assigns vehicles/drivers to routes based on optimized schedules and current fleet availability.
Notification & Alert Service	Messaging component	Subscribes to schedule, delay, and overcrowding events; pushes real-time alerts to passengers and operators.
Analytics & Reporting Service	Read-optimized component	Subscribes to all relevant events and consumes the data lake to generate ridership, performance, and utilization reports.

User / Auth Service	Identity component	Manages accounts, roles, and permissions; issues and validates access tokens used by the API Gateway.
Event Bus (Kafka/MQTT)	Messaging backbone	Decouples data producers (sensors, services) from consumers; enables real-time, asynchronous, many-to-many communication.
Data Ingestion & Integration Layer	Gateway/ETL component	Streams and cleans IoT/GPS telemetry; connects to external Weather and Traffic APIs; forwards standardized events to the bus.
Shared Data Platform	Data Lake, audit log, config store	Persists historical data, forecasts, audit trails, and configuration used across services for consistency and traceability.

4.2 Communication Mechanisms

- Synchronous communication (REST/HTTPS via the API Gateway) is used where an immediate response is required — e.g., a passenger requesting current arrival information, or an operator requesting a schedule view.
- Asynchronous communication (publish/subscribe via Kafka/MQTT) is used for continuous, high-volume, real-time data — e.g., GPS location updates, IoT occupancy readings, and cross-service notifications such as "schedule updated" or "overbooked route detected."
- Data read/write to the Shared Data Platform occurs both from the ingestion layer (writing raw/cleaned data) and from services such as Analytics (reading aggregated data), keeping transactional and analytical workloads separated.

4.3 Overall System Workflow

The workflow below traces a complete cycle: from raw sensor/API data entering the system, through AI forecasting and optimization, to real-time information reaching passengers and operators.

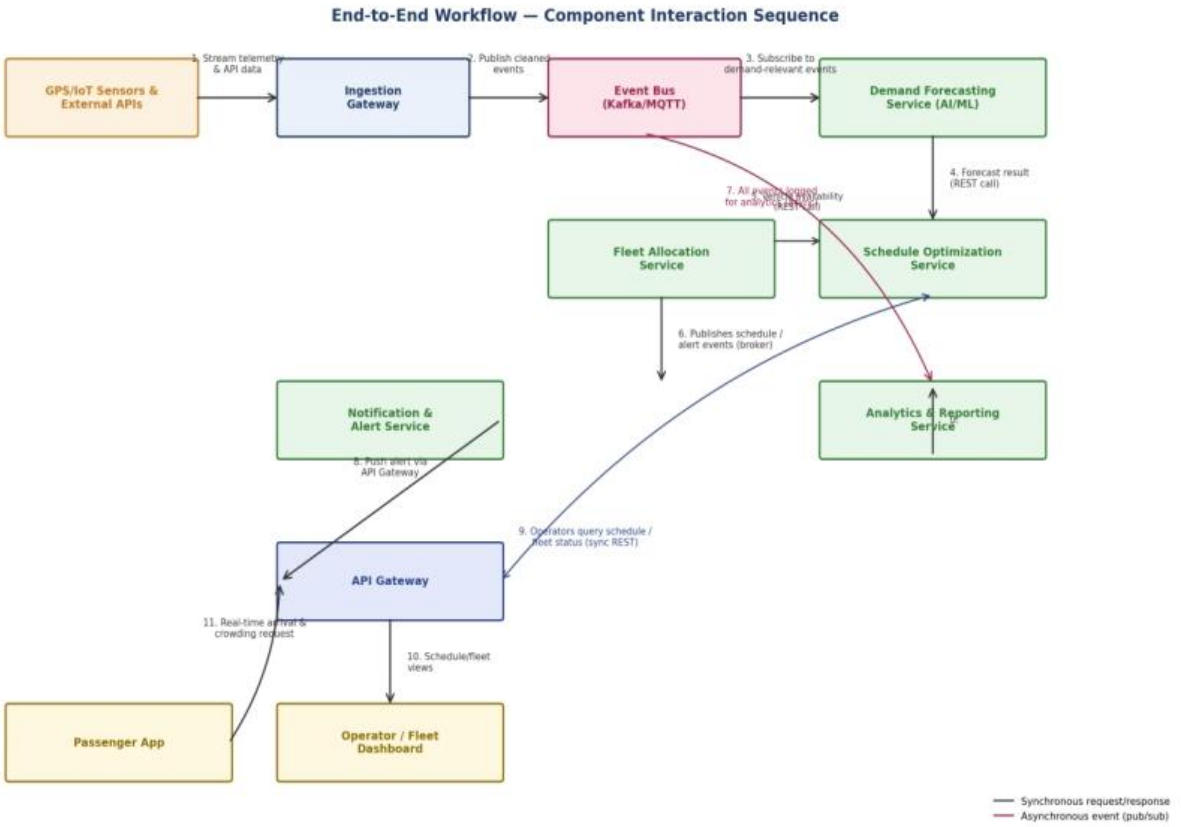

Figure 2: End-to-End Component Interaction / Workflow Sequence

1. GPS/IoT sensors and external weather/traffic APIs continuously stream raw data to the Ingestion Gateway.
2. The Ingestion Gateway cleans and standardizes the data, then publishes events to the Event Bus.
3. The Demand Forecasting Service subscribes to relevant events and generates updated demand predictions.
4. The Schedule Optimization Service retrieves the latest forecast (REST call) and computes an optimized schedule.
5. The Fleet Allocation Service checks vehicle availability and aligns it with the optimized schedule.
6. Schedule and fleet changes are published as events on the bus for any interested subscriber.
7. The Analytics & Reporting Service consumes all relevant events for historical trend analysis and reporting.
8. The Notification & Alert Service picks up relevant events and pushes alerts through the API Gateway.
9. Operators query live schedule and fleet status through the API Gateway (synchronous REST).

10. The API Gateway returns schedule/fleet views to the Operator/Fleet Dashboard.

11. Passengers request real-time arrival and crowding information through the same API Gateway.

5. Quality Attribute Discussion

The proposed hybrid architecture was evaluated against each of the key quality attributes identified in Section 1.3. The table below summarizes how specific architectural decisions address each attribute.

Quality Attribute	How the Architecture Addresses It
Scalability	Each microservice (forecasting, scheduling, fleet, notification, analytics) can be scaled horizontally and independently — for example, adding more Demand Forecasting instances during peak-hour prediction load without scaling the entire system. The event bus decouples producers and consumers, allowing thousands of IoT/GPS devices to publish data without overwhelming any single service.
Security	The API Gateway centralizes authentication and authorization using the User/Auth Service, enforcing role-based access control for passengers, operators, fleet managers, and admins. All client-service and service-service communication uses HTTPS/TLS, and sensitive data (e.g., passenger identifiers) is encrypted at rest in the Shared Data Platform.
Performance	Real-time passenger information is served through low-latency synchronous API calls, while high-volume telemetry is handled asynchronously so it never blocks user-facing requests. Caching of frequently requested forecasts and schedules at the API Gateway further reduces response times.
Reliability	Because services are independently deployed, a failure in a non-critical service (e.g., Analytics & Reporting) does not affect core passenger-facing functionality. The event bus provides message durability/replay, so temporary consumer downtime does not result in permanent data loss.
Maintainability	Each microservice has a clearly bounded responsibility and its own data store, so the AI forecasting model or optimization algorithm can be updated, tested, and redeployed independently of the rest of the system, reducing regression risk and deployment complexity.
Availability	Redundant instances of critical services (API Gateway, Notification Service, Demand Forecasting Service) behind load balancers, combined with the event bus's asynchronous decoupling, allow the system to keep functioning (in a degraded mode if necessary) even if one component is temporarily unavailable or being upgraded.

6. Justification of Architectural Decisions

6.1 Rationale Behind Key Choices

- Microservices over a monolith: chosen because the AI forecasting component has fundamentally different scaling and update cadence needs than, for example, the User/Auth service; a monolith would force all components to scale and redeploy together.
- Event-driven backbone over pure REST polling: chosen because continuous GPS/IoT telemetry arriving from thousands of vehicles is far more efficient and timely as a publish/subscribe stream than as repeated client polling.
- API Gateway over direct client-to-service calls: chosen to centralize authentication, simplify client development, and hide internal service topology, which also improves security and maintainability.
- Polyglot per-service data stores over one shared database: chosen so each service can use a data model best suited to it (e.g., a document/model store for the forecasting service vs. a relational store for scheduling), while the shared Data Lake preserves a unified historical view for analytics.

6.2 Trade-offs Considered

Trade-off	Decision Made and Reasoning
Microservices complexity vs. monolith simplicity	Accepted added operational complexity (service discovery, deployment orchestration) in exchange for independent scalability and fault isolation, which are essential given the system's mixed real-time and compute-heavy workloads.
Eventual consistency (event-driven) vs. strong consistency	Accepted eventual consistency for non-critical paths such as analytics and notifications, since a few seconds of delay is acceptable there, while keeping synchronous calls for operations that require an immediate, consistent response (e.g., fleet availability checks).
Infrastructure cost of a message broker vs. simpler point-to-point calls	Accepted the additional infrastructure (Kafka/MQTT cluster) because it is essential for reliably ingesting high-volume, high-frequency IoT/GPS data without tightly coupling every producer to every consumer.

6.3 Support for Future Enhancements and Integration

- New data sources (e.g., additional third-party mobility providers or ride-share integration) can be added as new publishers to the event bus without modifying existing services.

- New AI models or a completely re-engineered forecasting algorithm can be deployed as a new version of the Demand Forecasting Service without touching scheduling, fleet, or notification logic.
- Additional client applications (e.g., a future kiosk display or a partner mobile SDK) can be onboarded simply by registering with the API Gateway, without any backend redesign.
- The architecture supports incremental migration to serverless functions for bursty workloads (e.g., report generation) in the future, since each microservice is already independently deployable.

6.4 Long-Term Maintainability

Because responsibilities are clearly separated across services and layers, individual teams can own individual services, apply independent release cycles, and test in isolation. Combined with the event-driven backbone's loose coupling, this significantly reduces the long-term cost of extending or modifying the Intelligent AI-Based Smart Public Transportation Demand Forecasting System as requirements evolve.